The Cheat Code

Agent Patterns for Copilot Chat

↑ ↑ ↓ ↓ ← → ← → B A START

ISSUE #002 • MARCH 30, 2026 • WEEKLY

If your customer has deployed Copilot or any RAG-based agent and the results feel shallow or noisy, this issue is for you. The problem isn't usually the model — it's that the search is too broad.

This issue's pattern: How to make agent search dramatically better by scoping it to specific, known-relevant sources instead of searching everything.

🎮 AGENT SPOTLIGHT

The Work Item Triage Agent

Built by Raghav BN • Validated on live ADO work items

The scenario: Your team receives work items that need to be classified, cross-referenced against historical context, and routed. The triage person spends most of their time not on the item itself but on *searching for related information* — digging through Teams channels, past tickets, office hours notes, field readiness threads. That search step is where scoped search changes everything.

Raghav built an agent that reads an ADO work item and automatically searches across M365 sources to surface related context. The critical design choice: the agent doesn't search broadly. It first analyzes the work item, then generates *targeted queries* aimed at specific, known-relevant sources. The result: 80+ precise historical matches on a live Excel Copilot issue — context that would have taken a human 30 minutes to find manually.

🧩 **PATTERN BREAKDOWN**

Scoped Multi-Source Search

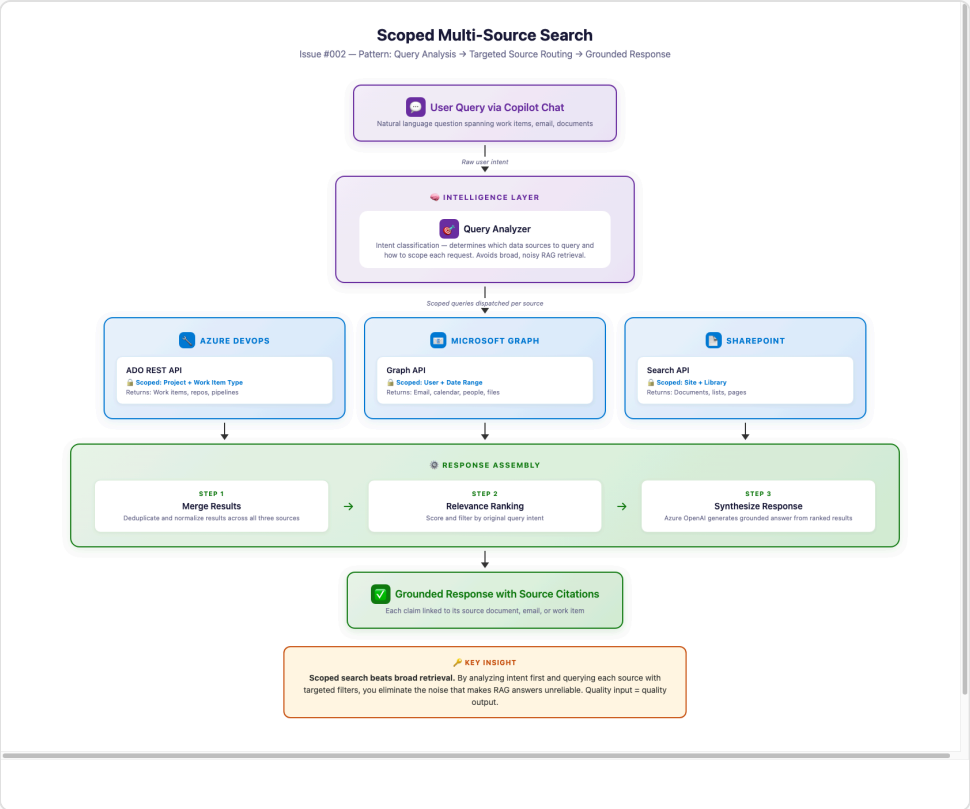
Stop searching everything. Search the right things.

Why broad search fails: When an agent searches "everything for anything related," it returns hundreds of results, most irrelevant. The model wastes tokens processing noise, and the output quality drops. This is the #1 complaint CSAs hear about RAG implementations: "Copilot found the wrong stuff."

The fix is a two-step process: First, analyze the input to understand what you're looking for. Then generate targeted search queries aimed at specific, known-relevant sources. The search is narrow by design.

✗ BROAD RAG	✓ SCOPED SEARCH
"Search all documents for anything about Excel Copilot accuracy" → Hundreds of results, most irrelevant	"Search #copilot-feedback for Excel calculation errors after March 2026" → 12 highly relevant threads

The two-step approach:



Scoped Multi-Source Search — query analysis routes to the right data, not all data

🔬 **Explore this pattern interactively →**

```
Analyze the input → extract key entities + intent
                          ↓
Generate scoped queries → target specific sources
                          ↓
Search only those sources → high-relevance results
```

What "scoping" actually means:

- **Source scoping** — search a specific Teams channel, not all of Teams
- **Time scoping** — search items from the last 90 days, not all time
- **Entity scoping** — search for the specific product, feature, or error type mentioned in the input
- **Role scoping** — search conversations from people who would know about this topic

Key insight: The first step in better search isn't improving the search engine. It's improving the *query*. When the agent analyzes the input before searching, the queries it generates are dramatically more precise than anything a human would type.

🛠️ **TRY THIS NOW**

Fix a Customer's Noisy Search in One Conversation

1. **Identify the complaint.** "Copilot isn't finding the right information" or "search results aren't relevant" — these are scoped search problems in disguise.
2. **Map the sources.** Ask: "When your best person handles this task, where do they actually look?" Write down the 3–5 specific sources. Those are your scoping targets.
3. **Add the analysis step.** Before the agent searches, it should first read the input and extract key entities, time ranges, and context. This turns a vague user question into a precise, multi-faceted query.

4. **Test the difference.** Run the same question through broad search and scoped search side by side. The quality gap is immediately obvious — and it's the best demo you can give.

💡 **Positioning:** "The model isn't the problem — the search is. Let me show you what happens when we point the agent at the right sources instead of everything."

🏆 **WHERE THIS PATTERN LANDS**

- **Any Copilot deployment** where users say "it doesn't find the right stuff"
- **Customer support** — search known KB articles and past tickets, not the whole SharePoint
- **Sales intelligence** — search CRM notes for a specific account, not all accounts
- **Compliance** — search relevant policy docs for a specific regulation, not the entire library
- **Engineering** — search incident history for a specific service, not all incidents

Bottom line: This pattern applies to every RAG-based agent you'll encounter. If the customer is unhappy with search quality, this is your first move. It's also the foundation for the more advanced triage patterns we'll cover in Issue #003.

Next issue: Prompt-Chained Triage + Playbook Orchestration

How to automate any read-classify-route workflow with configurable YAML playbooks.

The Cheat Code

ABS Tech Strategy • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Browse all issues at aka.ms/the-cheat-code

📦 Code sample — Coming Soon

Microsoft Internal • Not for external distribution